

Q1. const vs let vs var

Look at the code below. Circle every line that will cause an error and explain WHY it fails.

```
const name = 'Rahul';  
name = 'Priya'; // Line A  
let score = 10;  
let score = 20; // Line B  
var count = 1;  
var count = 2; // Line C  
const items = [1, 2, 3];  
items.push(4); // Line D  
items = [5, 6]; // Line E
```

■ Identify which lines error, and explain the difference between const, let, and var reassignment rules.

Your Answer:

Line A → **ERROR: TypeError: Assignment to constant variable.**

const variables cannot be reassigned once declared.

Line B → **ERROR: SyntaxError: Identifier 'score' has already been declared.**

let does not allow re-declaration in the same scope.

Line C → **NO ERROR.** var allows re-declaration in the same scope.

count simply becomes 2.

Line D → **NO ERROR.** items.push(4) mutates the array content.

const only prevents reassigning the variable, not mutating the object.

Line E → **ERROR: TypeError: Assignment to constant variable.**

items = [5, 6] tries to reassign the const variable to a new array.

Q2. Array Methods

Fill in the blanks. Write the correct output for each console.log below.

```
const nums = [3, 7, 1, 9, 4, 6];  
console.log(nums.length); // Output: 6  
console.log(nums[0]); // Output: 3  
console.log(nums[nums.length - 1]); // Output: 6  
const doubled = nums.map(n => n * 2);  
console.log(doubled); // Output: [6, 14, 2, 18, 8, 12]  
const big = nums.filter(n => n > 5);  
console.log(big); // Output: [7, 9, 6]  
const total = nums.reduce((acc, n) => acc + n, 0);  
console.log(total); // Output: 30
```

Your Answer:

nums.length → 6

nums[0] → 3

`nums[nums.length - 1] → 6 (nums[5] = last element)`
`doubled → [6, 14, 2, 18, 8, 12]`
`big → [7, 9, 6]`
`total → 30 (3+7+1+9+4+6 = 30)`

Q3. Objects & Dot Notation

Given the object below, write the JavaScript expression to access each value listed.

```
const user = {  
  name: 'Ankit',  
  age: 21,  
  scores: [85, 92, 78],  
  address: {  
    city: 'Pune',  
    pin: '411001'  
  }  
};  
  
// Write expressions to get:  
// 1. The user's name → user.name  
// 2. The user's city → user.address.city  
// 3. The second score (92) → user.scores[1]  
// 4. The pin code as number → Number(user.address.pin)  
// 5. Add a new key 'email' → user.email = 'ankit@example.com';
```

Your Answer:

1. `user.name`
 2. `user.address.city`
 3. `user.scores[1]`
 4. `Number(user.address.pin)`
 5. `user.email = 'ankit@example.com';`
-

Q4. Functions — Fix the Bug

The function below has 4 bugs. Find them, list them, and write the corrected version.

```
Function greet(name) {  
  const message = 'Hello, ' + Name + '!'  
  return  
  message  
}  
  
const result = greet('Meera')  
console.log(result)
```

■ *Hint: check capitalization, missing punctuation, and return statement placement.*

Your Answer:

- Bug 1:** 'Function' should be lowercase 'function'
- Bug 2:** 'Name' should be lowercase 'name' (matches parameter)

Bug 3: Missing semicolon after '!' in the message string

Bug 4: 'return' is on its own line – JS auto-inserts semicolon, so it returns undefined. Must be: return message;

// Corrected version:

```
function greet(name) {  
  const message = 'Hello, ' + name + '!';  
  return message;  
}  
  
const result = greet('Meera');  
console.log(result); // Output: Hello, Meera!
```

Q5. Write a Function — Predict & Code

Part A: What does the function below return when called with each input? Write the output. Part B: Write a new function reverseWords(str) that reverses the order of words in a sentence.

```
// Part A – Predict the output  
function mystery(arr) {  
  return arr  
    .filter(x => x % 2 === 0)  
    .map(x => x * x)  
    .reduce((sum, x) => sum + x, 0);  
}  
  
mystery([1, 2, 3, 4, 5]); // Output: 20  
mystery([10, 15, 20]); // Output: 500  
mystery([1, 3, 5]); // Output: 0
```

Your Answer:

```
mystery([1, 2, 3, 4, 5]):  
evens: [2, 4] → squared: [4, 16] → sum: 20  
mystery([10, 15, 20]):  
evens: [10, 20] → squared: [100, 400] → sum: 500  
mystery([1, 3, 5]):  
evens: [] → squared: [] → sum: 0 (no even numbers)
```

```
// Part B – Write reverseWords()  
// reverseWords('Hello World JS') → 'JS World Hello'  
// reverseWords('I love coding') → 'coding love I'
```

Write your code here:

```
function reverseWords(str) {  
  return str.split(' ').reverse().join(' ');  
}  
  
reverseWords('Hello World JS'); // → 'JS World Hello'  
reverseWords('I love coding'); // → 'coding love I'
```

Q6. Async / Await — Trace the Execution

Part A: Number the console.log lines in the ORDER they will actually print (1 = first, 4 = last). Part B: Rewrite the then/catch version as async/await.

```
// Part A – What order do these print?
console.log('A');
setTimeout(() => {
  console.log('B');
}, 0);
Promise.resolve().then(() => {
  console.log('C');
});
console.log('D');
// Order: A, D, C, B
```

Your Answer:

Order: A, D, C, B

Reason:

A → synchronous, runs first
D → synchronous, runs second
C → Promise micro-task, runs before macro-tasks
B → setTimeout macro-task, runs last

```
// Part B – Rewrite using async/await
function getData() {
  return fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(data => {
      console.log(data);
      return data;
    })
    .catch(err => {
      console.error('Failed:', err.message);
    });
}
```

Write your code here:

```
async function getData() {
  try {
    const res = await fetch('https://api.example.com/data');
    const data = await res.json();
    console.log(data);
    return data;
  } catch (err) {
    console.error('Failed:', err.message);
  }
}
```

Complete all three tasks using the JSON string provided.

```
const raw = `{
  "students": [
    { "name": "Arjun", "score": 88, "passed": true },
    { "name": "Sneha", "score": 45, "passed": false },
    { "name": "Vikram", "score": 72, "passed": true }
  ]
}`;

// Task 1: Parse raw into a JS object and print Sneha's score
// Task 2: Using the parsed object, filter only passed students
// and print their names as an array
// Task 3: Add a new student { name:'Pooja', score:91, passed:true }
// then JSON.stringify the whole updated object (pretty, 2 spaces)
// and print it
```

Write your code here:

```
// Task 1
const data = JSON.parse(raw);
const sneha = data.students.find(s => s.name === 'Sneha');
console.log(sneha.score); // Output: 45

// Task 2
const passedNames = data.students
  .filter(s => s.passed === true)
  .map(s => s.name);
console.log(passedNames); // Output: ['Arjun', 'Vikram']

// Task 3
data.students.push({ name: 'Pooja', score: 91, passed: true });
console.log(JSON.stringify(data, null, 2));
```

Q8. Error Handling — try / catch

Write a function `safeDivide(a, b)` and a function `parseAndDouble(jsonStr)` with proper error handling as described in the comments.

[illegible]

```
// If JSON is invalid, catch the error and return -1
// If 'value' key is missing, return -2
// Test calls:
// parseAndDouble('{\"value\": 21}') → 42
// parseAndDouble('not json') → -1
// parseAndDouble('{\"name\": \"hi\"}') → -2
```

Write your code here:

```
// Function 1
function safeDivide(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') {
    throw new Error('Inputs must be numbers');
  }
  if (b === 0) {
    throw new Error('Cannot divide by zero');
  }
  return a / b;
}

try { console.log('Result:', safeDivide(10, 2)); }
catch (e) { console.log('Error:', e.message); } // Result: 5

try { console.log('Result:', safeDivide(10, 0)); }
catch (e) { console.log('Error:', e.message); } // Error: Cannot divide by zero

try { console.log('Result:', safeDivide('x', 2)); }
catch (e) { console.log('Error:', e.message); } // Error: Inputs must be numbers

// Function 2
function parseAndDouble(jsonStr) {
  try {
    const obj = JSON.parse(jsonStr);
    if (!('value' in obj)) return -2;
    return obj.value * 2;
  } catch (e) {
    return -1;
  }
}

console.log(parseAndDouble('{\"value\": 21}')); // 42
console.log(parseAndDouble('not json')); // -1
console.log(parseAndDouble('{\"name\": \"hi\"}')); // -2
```